

1. Protocolo

Tecnológico Nacional de México

Instituto Tecnológico de León, Campus 1

Carrera: Ingeniería en Sistemas Computacionales

Materia: Estructura de Datos (Grupo B)

Horario: Lunes y miércoles de 8:45 a 10:25 hrs. y
Viernes de 8:45 a 9:35 hrs.

Alumnos: Marlene Inés Moreno Velázquez

Josue Abraham Segura Garcia

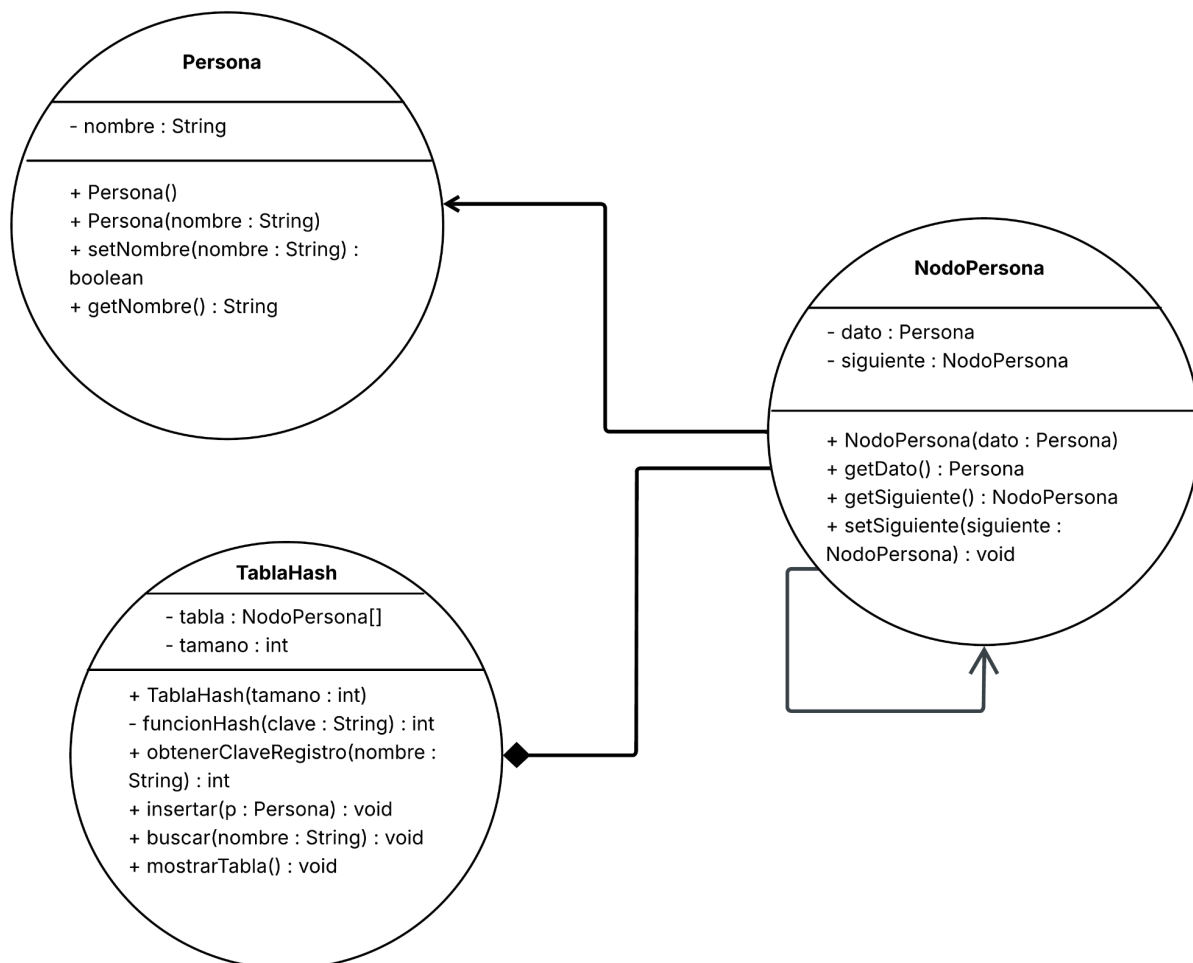
Número de lista: 12 y 17

Fecha: 8 de junio de 2026.

Exposicion: Hash Map



2. Diagrama UML



Tecnológico Nacional de México
Instituto Tecnológico de León, Campus 1
Carrera: Ingeniería en Sistemas Computacionales
Materia: Estructura de Datos (Grupo B)

Horario:

Lunes y miércoles de 8:45 a 10:25

Viernes de 8:45 a 9:35

Alumnos: Marlene Inés Moreno Velázquez y Josue Abraham Segura Garcia

Núm. de Lista: 12 y 17

Fecha: 10 de junio del 2026.

Exposición: HashMap

Tabla Hash ADT

Overview		
La estructura de datos TablaHash almacena y manipula colecciones de elementos asociándolos de forma eficiente a un índice único mapeado dentro de un arreglo estático residencial (cubetas). A diferencia de las estructuras secuenciales lineales puras, la Tabla Hash utiliza una función de dispersión matemática (aritmética modular sobre la suma de valores ASCII de una clave de texto) para determinar la ubicación directa de inserción y recuperación de los registros. Para mitigar y solucionar el fenómeno de colisiones de índices de cubetas idénticas, implementa una estrategia de resolución por encadenamiento, donde cada celda del arreglo actúa como cabecera de una estructura de lista ligada dinámica (NodoPersona), preservando el acceso óptimo sin dejar espacios inaccesibles o trunca la inserción de registros.		
Constructores		
Operación	Descripción	Ejemplo
TablaHash(tamano: int)	Inicializa un nuevo objeto de tipo TablaHash. Define la cantidad de cubetas totales de la tabla, instancia el arreglo físico de nodos e inicializa de manera explícita cada índice del arreglo en estado de referencia vacía (null).	TablaHash t = new TablaHash(11);
Acceso y Modificación		
Operación	Descripción	Ejemplo
obtenerClaveRegistro(nombre: String): int	Calcula y retorna de forma pública el valor entero del índice de la cubeta asignada para un elemento mediante el procesamiento interno de dispersión por suma ASCII.	int cubeta = t.obtenerClaveRegistro("Nombre");
insertar(p: Persona): void	Inserta un nuevo objeto Persona dentro de la estructura. Calcula su índice correspondiente; si la cubeta se	t.insertar(new Persona("Nombre"));

	encuentra vacía, lo asigna directamente como nodo inicial. Si existe una colisión, recorre la lista ligada de esa cubeta y lo enlaza dinámicamente al final de la secuencia.	
buscar(nombre: String): void	Localiza un registro dentro de la tabla hash. Accede de manera directa a la cubeta calculada por su dispersión y recorre de forma lineal únicamente los nodos encadenados en ella. Imprime directamente en la salida estándar el resultado del éxito o fallo, el número de la cubeta y la métrica de rendimiento medida en cantidad de saltos de punteros realizados.	t.buscar("Nombre");
mostrarTabla(): void	Recorre secuencialmente cada una de las posiciones o cubetas (desde 0 hasta tamaño - 1) y visualiza textualmente en consola el estado actual de la memoria. Muestra el nombre de los datos encapsulados apuntados en formato de secuencia ligada ([Dato] -> [Dato] -> null) o la palabra "Vacía" en caso de carecer de nodos.	t.mostrarTabla();
funcionHash(clave: String): int	Método Privado. Transforma la cadena de caracteres (clave) en un entero válido para el rango de la tabla. Itera sobre cada letra del texto sumando sus valores numéricos ASCII (int) y aplica la operación de residuo algebraico (%) sobre el atributo global tamaño.	(Invocación interna exclusiva de la clase)
Restricciones		
Variable u objeto	Descripción	
tamaño / capacidad : int	Debe ser estrictamente mayor a cero (tamaño > 0) al invocar el constructor primario para asegurar un dimensionamiento coherente del arreglo residente en memoria sin desbordamientos.	
p : Persona	El objeto pasado como argumento en la inserción no debe apuntar a una referencia nula (null), garantizando que la lectura interna de .getNombre() no propague una excepción en tiempo de ejecución.	
nombre / clave : String	La cadena de caracteres proporcionada para cálculos de dispersión o búsquedas debe ser válida y estar previamente formateada con texto real para evitar inconsistencias en el cálculo aritmético modular de la función hash.	

3. Códigos de las clases encapsuladas

```
// ===== Clase encapsulada Persona =====
package exposicion;

public class Persona {
    private String nombre;

    public Persona(){
        this.nombre = "";
    }

    public Persona(String nombre){
        this.nombre = nombre;
    }

    public boolean setNombre(String nombre){
        if(nombre == null || nombre.trim().isEmpty() || nombre.isBlank())
            return false;
        else if(!nombre.matches("[A-Za-zÁÉÍÓÚáéíóúñÑ ]+"))
            return false;
        else if(nombre.length() < 7 || nombre.length() > 50)
            return false;
        else {
            this.nombre = nombre.trim();
            return true;
        }
    }

    public String getNombre(){return nombre;}
}

// ===== Clase NodoPersona para las colisiones =====
package exposicion;

public class NodoPersona {
    private Persona dato;
    private NodoPersona siguiente;

    public NodoPersona(Persona dato) {
        this.dato = dato;
        this.siguiente = null;
    }

    public Persona getDato() {
        return dato;
    }
}
```

```

    public NodoPersona getSiguiente() {
        return siguiente;
    }

    public void setSiguiente(NodoPersona siguiente) {
        this.siguiente = siguiente;
    }
}

// ===== Clase TablaHash para el registro y búsqueda =====
package exposicion;

public class TablaHash {
    private NodoPersona[] tabla;
    private int tamano;

    public TablaHash(int tamano) {
        int i;
        this.tamano = tamano;
        this.tabla = new NodoPersona[tamano];

        for (i = 0; i < tamano; i++) tabla[i] = null;
    }

    private int funcionHash(String clave) {
        int suma;
        int i;
        char character;
        suma = 0;

        for (i = 0; i < clave.length(); i++) {
            character = clave.charAt(i);
            suma = suma + (int) character;
        }
        return suma % tamano;
    }

    public int obtenerClaveRegistro(String nombre) {
        return funcionHash(nombre);
    }

    public void insertar(Persona p) {
        int indice;
        NodoPersona nuevo;
        NodoPersona actual;

        indice = funcionHash(p.getNombre());
        nuevo = new NodoPersona(p);
    }
}

```

```

        if (tabla[indice] == null) {
            tabla[indice] = nuevo;
            return;
        }

        actual = tabla[indice];
        while (actual.getSiguiente() != null) {
            actual = actual.getSiguiente();
        }
        actual.setSiguiente(nuevo);
    }

    public void buscar(String nombre) {
        int indice;
        int comparaciones;
        NodoPersona actual;
        boolean encontrado;
        indice = funcionHash(nombre);
        comparaciones = 0;
        encontrado = false;
        actual = tabla[indice];

        System.out.println("Buscando [" + nombre + "] en cubeta: " + indice);

        while (actual != null) {
            comparaciones++;
            if (actual.getDato().getNombre().equals(nombre)) {
                encontrado = true;
                break;
            }
            actual = actual.getSiguiente();
        }
        if (encontrado) {
            System.out.println("EXITO: Encontrado.");
            System.out.println("-> Clave de registro (Cubeta): " + indice);
            System.out.println("-> Rendimiento: " + comparaciones
                + " saltos en la lista.");
        } else System.out.println("FALLO: No existe. Se realizaron "
            + comparaciones + " saltos.");
    }

    public void mostrarTabla() {
        int i;
        NodoPersona actual;

        System.out.println("\n--- ESTADO DE LA TABLA HASH ---");
        for (i = 0; i < tamano; i++) {
            System.out.print("Cubeta [" + i + "]: ");
            actual = tabla[i];

```

```

        if (actual == null) System.out.print("Vacia");
        while (actual != null) {
            System.out.print "["+actual.getDato().getNombre()+"] -> ";
            actual = actual.getSiguiente();
        }
        System.out.println("null");
    }
}

```

4. Algoritmo de la clase de prueba

1. Meta:

```

    Escribir("""
        ===== SISTEMA HASH - PRUEBA =====
        Programa que permite probar el ADT Tabla Hash manual,
        para registrar personas desde teclado o archivo, buscar
        nombres y visualizar la tabla con encadenamiento.
        =====
        """);

```

2. Data:

```

    Escribir("Inicializando Tabla Hash vacia con 11 cubetas...\n");
    Escribir("--- Estructura en memoria lista ---");

```

2.1 capturarNombre: String :

```

    Recibir atributo 'prompt'
    terminar mientras (verdadero) empezar
        Escribir(prompt);
        aux = ?;
        si (aux no está vacío) entonces
            retornar aux;
        sino entonces
            Escribir("Error: El nombre no puede estar vacio.");
        terminar si
    terminar mientras

```

3. Proceso:

```

    Recibir atributo 'option'
    segun (option) hacer
        caso 1:
            llamar a registrarManual();
            romper;
        caso 2:
            llamar a registrarDesdeArchivo();
            romper;
        caso 3:
            llamar a buscarPersona();

```

```

        romper;
    caso 4:
        Escribir("\nMostrando Estado de la Tabla:");
        llamar a tabla.mostrarTabla();
        romper;
    caso 5:
        llamar a vaciarTabla();
        romper;
terminar segun

```

3.1 registrarManual:

```

nuevaPersona = new Persona();
valido = falso;

terminar mientras (!valido) empezar
    Escribir("Ingrese el nombre de la persona: ");
    nombre = ?;

    si (llamar a nuevaPersona.setNombre(nombre)) entonces
        valido = verdadero;
    sino entonces
        Escribir("Error: Nombre invalido.");
    terminar si
terminar mientras

llamar a tabla.insertar(nuevaPersona);
Escribir("Registro insertado correctamente.");

```

3.2 registrarDesdeArchivo:

```

contadorExito = 0;
contadorFallos = 0;
Escribir("\nAbriendo selector de archivos...");

archivo = llamar a seleccionarArchivo();

si (archivo fue seleccionado) entonces
    intentar
        abrir archivo;
        terminar mientras (haya lineas por leer en el archivo) empezar
            linea = llamar a leerSiguienteLinea(archivo);

            si (linea no esta vacia) entonces
                si (llamar a validarNombre(linea)) entonces
                    llamar a tabla.insertar(linea);
                    contadorExito = contadorExito + 1;
                sino entonces
                    Escribir("Fallo de validacion. Se omitio: " + linea);
                    contadorFallos = contadorFallos + 1;
            terminar si
    terminar si

```



```

    terminar mientras
    cerrar archivo;

    Escribir("Proceso finalizado. Se cargaron " + contadorExito
        + " registros (" + contadorFallos + " omitidos).");
    capturar error empezar
    Escribir("Error: No se pudo leer el archivo correctamente.");
    terminar intentar
sino entonces
    Escribir("No se selecciono ningun archivo.");
terminar si

```

3.3 buscarPersona:

```

nombreBuscado = llamar a capturarNombre("Ingrese el nombre a buscar: ");
llamar a tabla.buscar(nombreBuscado);

```

3.4 vaciarTabla:

```

Escribir("=====");
Escribir("¡ADVERTENCIA! Se eliminaran todos los registros.");
Escribir("=====");
Escribir("¿Esta totalmente seguro de vaciar la tabla? (S/N): ");
confirmacion = ?;

si (confirmacion == 'S') entonces
    tabla = new TablaHash(11);
    Escribir("\nExito: La Tabla Hash ha sido vaciada por completo.");
sino entonces
    Escribir("\nOperacion cancelada. Los registros siguen a salvo.");
terminar si

```

4. Resultados:

```

Escribir("Finalizando el sistema...");

```

5. Navegabilidad:

```

llamar a meta();
llamar a data();
opcion = 0;

terminar mientras (opcion != 6) empezar
    opcion = llamar a menuPrincipal();
    si (opcion != 6) entonces
        llamar a proceso(opcion);
    terminar si
terminar mientras
llamar a resultados();

```

5.1 menuPrincipal: int :

```

opcion = 0;
valido = falso;

```

```

terminar mientras (!valido) empezar
    Escribir("""
        OPCIONES DE TABLA HASH:
        1) Registrar persona (Teclado)
        2) Registrar personas (Archivo TXT)
        3) Buscar persona por nombre
        4) Mostrar estado de la Tabla Hash
        5) Vaciar todos los registros
        6) Salir
        Su eleccion >>> """);
    aux = ?;
    intentar
        opcion = llamar a convertirEntero(aux);
        si (opcion >= 1 y opcion <= 6) entonces
            valido = verdadero;
        sino entonces
            Escribir("Error: Ingrese un numero (1 al 6).");
        terminar si
    capturar error e empezar
        Escribir("Error: Ingrese unicamente un valor numerico.");
    terminar intentar
terminar mientras
retornar opcion;

```

5. Código de Clase de Prueba ProgramaHash

```

package exposicion;

import java.io.BufferedReader;
import java.io.File;
import java.io.FileReader;
import java.io.IOException;
import java.util.Scanner;
import javax.swing.JFileChooser;

public class ProgramaHash {
    private int size = 11;
    private TablaHash tabla = new TablaHash(size);
    private Scanner sc = new Scanner(System.in);

    void meta() {
        String mensaje = ""
            + "===== SISTEMA HASH - PRUEBA ====="
            + "Programa que permite probar el ADT Tabla Hash manual,"
            + "para registrar personas desde teclado o archivo, buscar"
            + "nombres y visualizar la tabla con encadenamiento."
            + "===== "
            + "";
    }
}

```

```

        System.out.println(mensaje);
    }

    void data() {
        System.out.println("Inicializando Tabla Hash vacia con "
            + size + "cubetas\n");
        System.out.println("--- Estructura en memoria lista ---");
    }

    String capturarNombre(String prompt) {
        String aux;
        while (true) {
            System.out.print(prompt);
            aux = sc.nextLine();
            if (!aux.trim().isEmpty()) return aux.trim();
            System.out.println("Error: El nombre no puede estar vacio.");
        }
    }

    void proceso(int option) {
        switch(option) {
            case 1 -> registrarManual();
            case 2 -> registrarDesdeArchivo();
            case 3 -> buscarPersona();
            case 4 -> {
                System.out.println("\nMostrando Estado de la Tabla:");
                tabla.mostrarTabla(); }
            case 5 -> vaciarTabla();
        }
    }

    void registrarManual() {
        Persona nuevaPersona = new Persona();
        boolean valido = false;

        while (!valido) {
            System.out.print("Ingrese el nombre de la persona: ");
            String nombre = sc.nextLine();
            String msjE = ""
                + "Error: Nombre invalido. Asegurese de que tenga\n"
                + "entre 7 y 50 caracteres y contenga unicamente\n"
                + "letras.";

            if (nuevaPersona.setNombre(nombre)) valido = true;
            else System.out.println(msjE);
        }
        tabla.insertar(nuevaPersona);
        System.out.println("Registro insertado correctamente.");
    }
}

```

```

void registrarDesdeArchivo() {
    JFileChooser selector = new JFileChooser();
    File archivo; FileReader lector; BufferedReader buffer; String linea;
    int contadorExito = 0, contadorFallos = 0;

    System.out.println("\nAbriendo selector de archivos...");
    if (selector.showOpenDialog(null) == JFileChooser.APPROVE_OPTION) {
        archivo = selector.getSelectedFile();
        try {
            lector = new FileReader(archivo);
            buffer = new BufferedReader(lector);
            linea = buffer.readLine();

            while (linea != null) {
                if (!linea.trim().isEmpty()) {
                    Persona nuevaPersona = new Persona();
                    if (nuevaPersona.setNombre(linea)) {
                        tabla.insertar(nuevaPersona);
                        contadorExito++;
                    } else {
                        String msjE = ""
                                + "Fallo de validacion: Se omitio el "
                                + "registro invalido: " + linea.trim() + ". ";
                        System.out.println(msjE + linea.trim() + ".");
                        contadorFallos++;
                    }
                }
                linea = buffer.readLine();
            }
            buffer.close();
            System.out.println("Proceso finalizado. Se cargaron "
                    + contadorExito + " registros correctamente ("
                    + contadorFallos + " omitidos).");
        } catch (IOException e) {
            System.out.println("Error al leer archivo: "
                    + e.getMessage());
        }
    } else System.out.println("No se selecciono ningun archivo.");
}

void buscarPersona() {
    String nombreB = capturarNombre("\nIngrese el nombre a buscar: ");
    tabla.buscar(nombreB);
}

void vaciarTabla() {
    System.out.println("¡ADVERTENCIA! Se eliminaran todos los "
            + "registros.");
    System.out.print("¿Esta totalmente seguro de vaciar la tabla?"
            + "(S/N): ");
    String confirmacion = sc.nextLine().trim().toUpperCase();
}

```

```

        if (confirmacion.equalsIgnoreCase("S")) {
            tabla = new TablaHash(size);
            System.out.println("\nLa Tabla Hash ha sido vaciada.");
        } else System.out.println("\nOperacion cancelada.");
    }

    void resultados() {
        System.out.println("Finalizando el sistema...");
    }

    void navegabilidad() {
        meta();
        data();
        int opcion = 0;

        while (opcion != 6) {
            opcion = menuPrincipal();
            if (opcion != 6) proceso(opcion);
        }
        resultados();
        sc.close();
    }

    int menuPrincipal() {
        String menu = ""
            \nOPCIONES DE TABLA HASH:
            1) Registrar persona (Teclado)
            2) Registrar personas (Archivo TXT)
            3) Buscar persona por nombre
            4) Mostrar estado de la Tabla Hash
            5) Vaciar todos los registros
            6) Salir
            "";
        int opcion = 0;
        boolean valido = false;

        while (!valido) {
            System.out.print(menu + "Su eleccion >>> ");
            String aux = sc.nextLine();
            try {
                opcion = Integer.parseInt(aux);
                if (opcion >= 1 && opcion <= 6) valido = true;
                else System.out.println("Error: Ingrese un numero (1 - 6).");
            } catch (NumberFormatException e) {
                System.out.println("Error: Ingrese solamente un numero.");
            }
        }
        return opcion;
    }
}

```

```

    public static void main(String[] args) {
        ProgramaHash programa = new ProgramaHash();
        programa.navegabilidad();
    }
}

```

6. Resultados de corrida

===== SISTEMA HASH - PRUEBA =====
 Programa que permite probar el ADT Tabla Hash manual,
 para registrar personas desde teclado o archivo, buscar
 nombres y visualizar la tabla con encadenamiento.
 =====

Inicializando Tabla Hash vacia con 11 cubetas

--- Estructura en memoria lista ---

OPCIONES DE TABLA HASH:

- 1) Registrar persona (Teclado)
- 2) Registrar personas (Archivo TXT)
- 3) Buscar persona por nombre
- 4) Mostrar estado de la Tabla Hash
- 5) Vaciar todos los registros
- 6) Salir

Su eleccion >>> 1

Ingrese el nombre de la persona: **SILVA TOVAR CIRINO**

Registro insertado correctamente.

OPCIONES DE TABLA HASH:

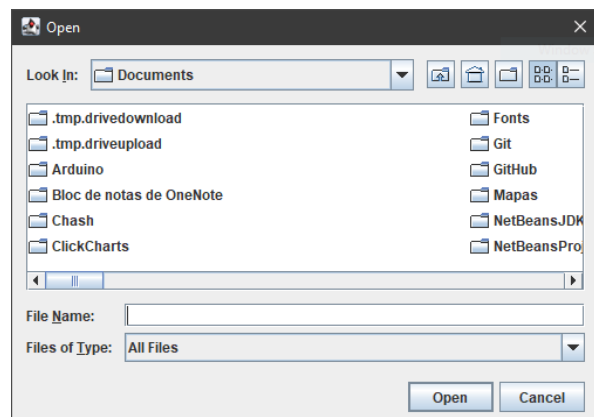
- 1) Registrar persona (Teclado)
- 2) Registrar personas (Archivo TXT)
- 3) Buscar persona por nombre
- 4) Mostrar estado de la Tabla Hash
- 5) Vaciar todos los registros
- 6) Salir

Su eleccion >>> 2

Abriendo selector de archivos...

*[Se abre una ventana selector de
archivos para que el usuario elija el
archivo de datos fuente]*

Proceso finalizado. Se cargaron 19
registros correctamente (0 omitidos).



OPCIONES DE TABLA HASH:

- 1) Registrar persona (Teclado)
- 2) Registrar personas (Archivo TXT)
- 3) Buscar persona por nombre
- 4) Mostrar estado de la Tabla Hash
- 5) Vaciar todos los registros
- 6) Salir

Su eleccion >>> 4

Mostrando Estado de la Tabla:

--- ESTADO DE LA TABLA HASH ---

Cubeta [0]: [MORENO VELAZQUEZ MARLENE INES] ->
[PALOMINO CERVANTES MAURICIO MAGDIEL] ->
[SANCHEZ PEREZ CRISTIAN EMMANUEL] -> null
Cubeta [1]: [CASTRO PONCE JESUS OSWALDO] -> null
Cubeta [2]: [GARCIA TORRES BRYAN ORLANDO] -> [ROMERO NUÑEZ LUCIANO] -> null
Cubeta [3]: [GARCIA OLVERA JUAN CARLOS] -> [GARDUÑO ZAVALA JORGE EDUARDO] ->
null
Cubeta [4]: [BECERRA ANDRADE ERIKA SUGEY] -> [VILCHIS MOLINA MAURICIO] ->
null
Cubeta [5]: null
Cubeta [6]: [GUERRERO FRAUSTO PATRICIO GERARDO] -> null
Cubeta [7]: [GARCIA GROWER LUIS ZAID] -> null
Cubeta [8]: [SILVA TOVAR CIRINO] -> [GONZALEZ MEDINA LIZETTE CAROLINA] ->
[SEGURA GARCIA JOSUE ABRAHAM] -> [VALDEZ ARIAS JAZIEL ULISES] ->
null
Cubeta [9]: [ARREDONDO ESCOBAR JUAN PABLO] ->
[CAUDILLO SOTO JESUS EMMANUEL] ->
[MARTINEZ TRUJILLO LEONEL] ->
[RAMIREZ AGUIRRE ERICK RAUL] -> null
Cubeta [10]: null

OPCIONES DE TABLA HASH:

- 1) Registrar persona (Teclado)
- 2) Registrar personas (Archivo TXT)
- 3) Buscar persona por nombre
- 4) Mostrar estado de la Tabla Hash
- 5) Vaciar todos los registros
- 6) Salir

Su eleccion >>> 3

Ingrese el nombre a buscar: **MORENO VELAZQUEZ MARLENE INES**

Buscando [MORENO VELAZQUEZ MARLENE INES] en cubeta: 0

EXITO: Encontrado.

-> Clave de registro (Cubeta): 0

-> Rendimiento: 1 saltos en la lista.

OPCIONES DE TABLA HASH:

- 1) Registrar persona (Teclado)
- 2) Registrar personas (Archivo TXT)
- 3) Buscar persona por nombre
- 4) Mostrar estado de la Tabla Hash
- 5) Vaciar todos los registros
- 6) Salir

Su eleccion >>> 3

Ingrese el nombre a buscar: **SEGURA GARCIA JOSUE ABRAHAM**

Buscando [SEGURA GARCIA JOSUE ABRAHAM] en cubeta: 8

EXITO: Encontrado.

-> Clave de registro (Cubeta): 8

-> Rendimiento: 3 saltos en la lista.

OPCIONES DE TABLA HASH:

- 1) Registrar persona (Teclado)
- 2) Registrar personas (Archivo TXT)
- 3) Buscar persona por nombre
- 4) Mostrar estado de la Tabla Hash
- 5) Vaciar todos los registros
- 6) Salir

Su eleccion >>> 10

Error: Ingrese un numero (1 al 6).

OPCIONES DE TABLA HASH:

- 1) Registrar persona (Teclado)
- 2) Registrar personas (Archivo TXT)
- 3) Buscar persona por nombre
- 4) Mostrar estado de la Tabla Hash
- 5) Vaciar todos los registros
- 6) Salir

Su eleccion >>> no lo se

Error: Ingrese solamente un valor numerico.

OPCIONES DE TABLA HASH:

- 1) Registrar persona (Teclado)
- 2) Registrar personas (Archivo TXT)
- 3) Buscar persona por nombre
- 4) Mostrar estado de la Tabla Hash
- 5) Vaciar todos los registros
- 6) Salir

Su eleccion >>> 1

Ingrese el nombre de la persona: **25240001**

Error: Nombre invalido. Asegurese de que tenga entre 7 y 50 caracteres y contenga unicamente letras.

Ingrese el nombre de la persona: **GONZALES GONZALO**

Registro insertado correctamente.

OPCIONES DE TABLA HASH:

- 1) Registrar persona (Teclado)
- 2) Registrar personas (Archivo TXT)
- 3) Buscar persona por nombre
- 4) Mostrar estado de la Tabla Hash
- 5) Vaciar todos los registros
- 6) Salir

Su eleccion >>> 5

=====

◆ADVERTENCIA! Se eliminaran todos los registros.

=====

◆Esta totalmente seguro de vaciar la tabla? (S/N): n

Operacion cancelada. Los registros siguen a salvo.

OPCIONES DE TABLA HASH:

- 1) Registrar persona (Teclado)
- 2) Registrar personas (Archivo TXT)
- 3) Buscar persona por nombre
- 4) Mostrar estado de la Tabla Hash
- 5) Vaciar todos los registros
- 6) Salir

Su eleccion >>> 5

=====

◆ADVERTENCIA! Se eliminaran todos los registros.

=====

◆Esta totalmente seguro de vaciar la tabla? (S/N): s

Exito: La Tabla Hash ha sido vaciada por completo.

OPCIONES DE TABLA HASH:

- 1) Registrar persona (Teclado)
- 2) Registrar personas (Archivo TXT)
- 3) Buscar persona por nombre
- 4) Mostrar estado de la Tabla Hash
- 5) Vaciar todos los registros
- 6) Salir

Su eleccion >>> 4

Mostrando Estado de la Tabla:

--- ESTADO DE LA TABLA HASH ---

Cubeta [0]: Vacianull

Cubeta [1]: Vacianull

Cubeta [2]: Vacianull

Cubeta [3]: Vacianull

Cubeta [4]: Vacianull

Cubeta [5]: Vacianull

Cubeta [6]: Vacianull
Cubeta [7]: Vacianull
Cubeta [8]: Vacianull
Cubeta [9]: Vacianull
Cubeta [10]: Vacianull

OPCIONES DE TABLA HASH:

- 1) Registrar persona (Teclado)
 - 2) Registrar personas (Archivo TXT)
 - 3) Buscar persona por nombre
 - 4) Mostrar estado de la Tabla Hash
 - 5) Vaciar todos los registros
 - 6) Salir
- Su eleccion >>> 6
Finalizando el sistema...
-

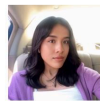
7. Diapositivas de la presentación



TECNOLOGICO NACIONAL DE MEXICO
INSTITUTO TECNOLÓGICO DEL LEON CAMPUS 1
INGENIERIA EN SISTEMAS COMPUTACIONALES
ESTRUCTURA DE DATOS
8:45 - 10:30
03/06/2026



Hash Map



MARLENE INES MORENO VELAZQUEZ
JOSUE ABRAHAM SEGURA GARCIA

Un **hash map** es una estructura de datos que almacena información en pares de clave-valor. Usa una función matemática llamada "hash" para transformar la clave en una posición de memoria.

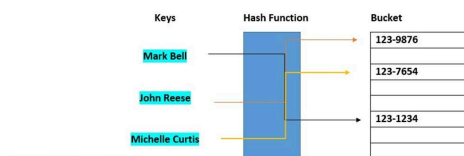
Propuesto por Hans Peter Luhn en enero de 1953. Mientras trabajaba como investigador en IBM, Luhn escribió un memorándum interno donde planteaba un nuevo método para agilizar la búsqueda de información.



Hans Peter Luhn

LA FUNCIÓN HASH

Es el núcleo lógico de la estructura. Su trabajo es tomar una llave de entrada (*por ejemplo, una cadena de texto*) y transformarla mediante un cálculo matemático en un índice numérico que corresponde a una posición dentro de un arreglo.



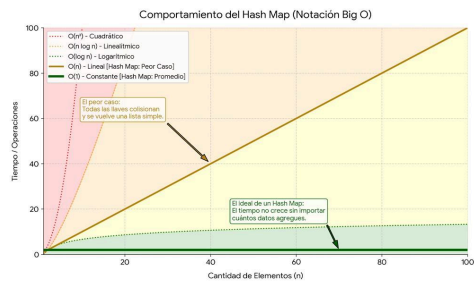
Funcionamiento básico en la asignación de memoria

EFICIENCIA

La gran ventaja de esta estructura frente a las listas simples o los árboles es su velocidad de acceso.

- **Mejor caso y promedio:** Las operaciones de búsqueda, inserción y eliminación se ejecutan en tiempo constante de $O(1)$.
- **Peor caso:** Si la función hash es ineficiente y envía todos los datos al mismo índice, la complejidad se degrada a $O(n)$.

EFICIENCIA



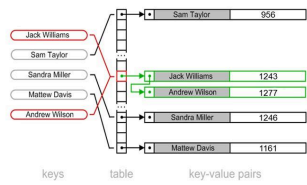
COMPARACION DE COMPLEJIDADES DE UNA BUSQUEDA POR FUNCION HASH CON OTROS METODOS DE BUSQUEDA

Método	Complejidad
Secuencial	O(n)
Binaria	O(log n)
Hash (ideal)	O(1)
Hash (muchas colisiones)	O(n)

MANEJO DE COLISIONES

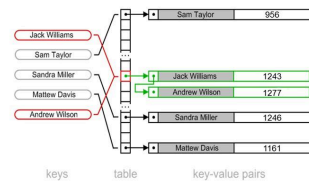
Dado que el arreglo en memoria tiene un tamaño finito, es inevitable que dos claves completamente distintas arrojen el mismo índice matemático. A este evento se le llama **colisión**. Los dos métodos principales para resolverlo son:

1. Encadenamiento (Separate Chaining): Cada posición del arreglo contiene un apuntador hacia una lista enlazada. Si hay una colisión, el nuevo dato simplemente se añade como un nodo extra en esa lista. (Este fue el método propuesto originalmente por Luhn).



MANEJO DE COLISIONES

2. Direccionamiento Abierto (Open Addressing): Todos los elementos se guardan en el mismo arreglo. Si la casilla calculada ya está ocupada, el algoritmo inicia una secuencia de prueba (*probing*) para buscar y ocupar la siguiente casilla vacía disponible.



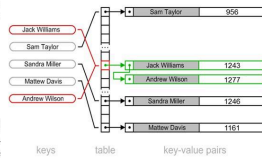
CREACION DE CLAVES

Para mapear cadenas de texto (nombres) a índices numéricos de un arreglo de tamaño fijo, utilizaremos el método de suma ponderada de caracteres seguido del método de la división.

Matemáticamente, la función se define como:

$$h(K) = \left(\sum_{i=0}^{n-1} \text{ASCII}(K_i) \right) \pmod{M}$$

Donde K es la clave (nombre), n es la longitud de la cadena y M es el tamaño de la tabla. El operador módulo asegura que el índice resultante siempre esté dentro de los límites del arreglo $[0, M - 1]$.

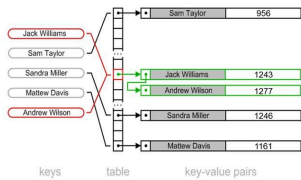


SCRIPT EN PYTHON PARA LA CREACION DE CLAVES

Resultados de la corrida en la consola de depuración:

ASCII sums:
Jack Williams: 1243
Sam Taylor: 956
Sandra Miller: 1246
Matthew Davis: 1161

Con estos resultados podemos calcular el módulo de la suma ASCII entre el número de celdas o lugares disponibles y así obtener la posición del arreglo.



IMPLEMENTACION DE BUSQUEDA POR FUNCION HASH

Ej. Ficha de Datos De Una Persona

Para la implementación del algoritmo de búsqueda mediante funciones Hash, se utilizarán objetos de tipo Persona, clase encapsulada previamente empleada en los algoritmos de búsqueda secuencial y búsqueda binaria.

Esto permitirá realizar una comparación directa del comportamiento, eficiencia y tiempo de ejecución entre los distintos métodos de búsqueda estudiados.

Numero de control	25240001
Nombre	Gonzales Castro Juan Diego
Fecha de Nacimiento	07/02/2006
Peso	74 kg
Estatura	1.78 m

IMPLEMENTACION DE BUSQUEDA POR FUNCION HASH

Ej. Ficha de Datos De Una Persona

El proceso requiere identificar los atributos almacenados en cada registro de la estructura Persona, con el propósito de seleccionar uno que funcione como clave única o identificador principal dentro de la tabla Hash.

Numero de control	25240001
Nombre	Gonzales Castro Juan Diego
Fecha de Nacimiento	07/02/2006
Peso	74 kg
Estatura	1.78 m

IMPLEMENTACION DE BUSQUEDA POR FUNCION HASH

Numero de control	25240001
Nombre	Gonzales Castro Juan Diego
Fecha de Nacimiento	07/02/2006
Peso	74 kg
Estatura	1.78 m

Ej. Ficha de Datos De Una Persona

En este proyecto se empleará el número de control como **clave de búsqueda**, debido a que representa un dato con alta probabilidad de unicidad y facilita una distribución más uniforme dentro de la tabla Hash.

No obstante, otros atributos, como el nombre, también podrían utilizarse como clave, aunque esto incrementaría la posibilidad de **colisiones** y reduciría la eficiencia del proceso de búsqueda.

IMPLEMENTACION DE BUSQUEDA POR FUNCION HASH

Numero de control	25240001
Nombre	Gonzales Castro Juan Diego
Fecha de Nacimiento	07/02/2006
Peso	74 kg
Estatura	1.78 m

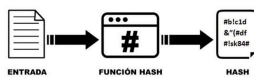
Ej. Ficha de Datos De Una Persona

A partir de dicha clave, una función Hash transformará el valor original en una posición específica dentro de la tabla, permitiendo acceder a la información deseada con tiempos de búsqueda cercanos a $O(1)$ en condiciones ideales.

PARTE MEDULAR DE UNA BUSQUEDA POR FUNCION HASH

La parte medular de una búsqueda Hash son estas 3 operaciones:

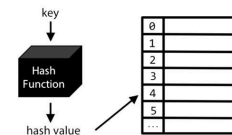
1. Función Hash → transforma la clave en índice
2. Inserción → guarda en la cubeta
3. Búsqueda → localiza usando la misma función hash



A continuación, se mostrará la implementación manual de **encadenamiento por listas enlazadas**.

REFERENCIAS BIBLIOGRAFICAS

1. Databricks Staff. (s.f.). *What are Hash Buckets?* **Databricks**. Recuperado el 1 de junio de 2026, de <https://www.databricks.com/blog/what-are-hash-buckets>
2. DataCamp. (s.f.). *Java HashMap*. **DataCamp**. Recuperado el 1 de junio de 2026, de <https://www.datacamp.com/es/doc/java/hashmap>
3. Luhn, H. P. (1953). *A New Method of Organizing Information through Key-to-Address Transformations*. **IBM Internal Memorandum**. Recuperado el 1 de junio de 2026, de <https://cs.pomona.edu/classes/cs62/history/hashtables/>



7. Referencias Bibliográficas

1. Databricks Staff. (s.f.). *What are Hash Buckets?* **Databricks**. Recuperado el 1 de junio de 2026, de <https://www.databricks.com/blog/w4hat-are-hash-buckets>
2. DataCamp. (s.f.). *Java HashMap*. **DataCamp**. Recuperado el 1 de junio de 2026, de <https://www.datacamp.com/es/doc/java/hashmap>
3. Luhn, H. P. (1953). *A New Method of Organizing Information through Key-to-Address Transformations*. **IBM Internal Memorandum**. Recuperado el 1 de junio de 2026, de <https://cs.pomona.edu/classes/cs62/history/hashtables/>
4. Recursos de la exposición:
 - a. Repositorio de GitHub
<https://github.com/marinmov/Estructura-de-Datos/tree/54bc5bd89ff0825d8be684bf06f51f9218f60e9d/src/main/java/exposicion>
 - b. Drive con los recursos
https://drive.google.com/drive/folders/177II8VZJ1Wfqkiuc7J6vHd_gBn0lexd2?usp=drive_link